A3-PS003-TC517

Federated Traffic – Flow Prediction Project Guidance

Adamya Srivastava
adamyasrivastava30@gmail.com

AIORI Hackathon 3

October 14, 2026

Abstract

Problem Description

A Federated Traffic-Flow Prediction with Communication-Cost Evaluation

The objective of this project is to build a **Federated Learning (FL) simulator** for traffic forecasting, contrasting centralized learning against federated setups across simulated intersections under both **IID (Independent and Identically Distributed)** and **non-IID** data distributions.

Background

1. Adamya Srivastava A3-PS003-TC517 October 14,2026

Background

- **Urban Growth & ITS:** Modern Intelligent Transportation Systems (ITS) rely heavily on accurate traffic-flow prediction to manage congestion and optimize urban mobility.

- **Centralized Limitations:** Traditional cloud-based forecasting models create severe privacy risks by collecting raw sensor data and imposing heavy network bandwidth bottlenecks.

- **Federated Learning Solution:** Federated Learning (FL) resolves privacy issues by enabling distributed intersections to train local models collaboratively without exposing raw local logs.

- **The Communication Challenge:** Frequent parameter exchanges between edge nodes and the server create network overhead, further complicated by heterogeneous (non-IID) traffic patterns across intersections.

- **Project Scope (PS-003):** This project simulates a multi-client FL traffic forecasting environment using FedAvg to evaluate prediction accuracy against communication costs under both IID and non-IID conditions.

Literature & RFC Review

Literature & RFC Review

Evolution of Traffic-Flow Prediction Models

- **From Centralized to Distributed:** Early traffic forecasting relied on centralized historical data aggregation using statistical models (e.g., ARIMA) and deep spatial-temporal architectures (e.g., STGCN, TCN). However, recent literature highlights severe vulnerabilities regarding user location privacy and network ingestion bottlenecks when routing raw data to the cloud.
- **Federated Learning (FL) in Intelligent Transportation:** Recent studies (such as FedTraffic and personalized FL frameworks) prove that collaborative training across

decentralized nodes (intersections/base stations) successfully preserves local spatial context without transferring raw logs.

Communication-Cost and Heterogeneity Challenges

- **The Non-IID Drift:** Urban intersections exhibit heavy data heterogeneity (non-IID distribution) due to distinct localized geographical patterns (e.g., residential vs. commercial traffic peaks). Literature confirms that standard **Federated Averaging (FedAvg)** suffers from client drift under non-IID conditions, requiring careful evaluation of convergence rounds.
- **Overhead Mitigation:** To counter heavy uplink/downlink communication costs, cutting-edge studies explore lightweight parameter optimization, model pruning, and quantization to minimize payload sizes transmitted during aggregation rounds.

.

Proposed Solution

The proposed solution for **PS-003** is a lightweight **Federated Learning (FL) Simulation Framework** designed to predict traffic flow across distributed intersections while explicitly tracking and evaluating communication overhead.

The core architecture and workflow consist of the following key components:

Multi-Client Edge Architecture: Simulates 3 to 5 virtual traffic intersection nodes connected to a central aggregation server, mimicking a decentralized smart city setup without requiring physical hardware.

Dual Data Distribution Scenarios:

IID Setup: Data is randomly distributed across clients to establish a performance baseline.

Non-IID Setup: Data is split to reflect real-world urban heterogeneity (e.g., specific intersections handling heavy morning rushes vs. night-time lulls) to test model robustness and client drift.

Communication-Cost Evaluation Metric: Integrates a serialization payload tracker to measure exact uplink and down-link network traffic (in bytes) per training round, correlating

network expenditure against prediction accuracy and convergence speed across **3 random seeds**.

Optimization Proposed

1. **Model Weight Quantization:**

- *Description:* Converts 32-bit floating-point parameters (float32) to 16-bit (float16) or 8-bit integers (int8) before serialization.

- *Impact:* Dramatically cuts down the byte size of uplink and downlink payloads without sacrificing prediction accuracy.

2. **Algorithmic Regularization (FedProx Integration):**

   - *Description:* Adds a proximal term to the local objective function to limit model drift across heterogeneous (non-IID) intersections.

- *Impact:* Stabilizes convergence and reduces the total number of communication rounds required.

3. **Adaptive Local Epoch Scaling:**

- *Description:* Allows edge clients to run multiple thorough local training epochs before initiating global synchronization.

- *Impact:* Minimizes communication frequency and limits network congestion.

4. **Selective Client Sampling:**

- *Description:* Involves only a representative subset of virtual intersections per training round instead of forcing all nodes to transmit updates simultaneously.

- *Impact:* Lowers concurrent server-side bandwidth pressure and speeds up iteration loops.

Before vs. After Optimization Comparison Table

Before vs. After Optimization Comparison

Baseline FedAvg compared with the optimized federated-learning workflow

Feature / Metric	Baseline Scenario (Standard FedAvg)	Optimized Scenario (With Enhancements)
Payload Format & Size	Uncompressed 32-bit (float32) weights; high byte count per round.	Quantized 16-bit or 8-bit weights; approximately 50%–75% reduction in transmission size.
Handling of Non-IID Data	Prone to client drift, unstable training gradients, and slow convergence.	Regularized via a local proximal term, encouraging steadier and more predictable convergence.
Communication Frequency	Synchronizes after every single epoch or after minimal local computation.	Uses extended local training and selective client sampling to reduce synchronization frequency.
		Reduced network expenditure;

Payload Format & Size

Model weight quantization

Baseline scenario

The baseline sends uncompressed 32-bit floating-point (float32) model weights. This creates a relatively high byte count for each communication round.

Optimized scenario

Quantization converts float32 parameters to float16 or int8 before serialization. The expected payload-size reduction is approximately 50% to 75%, depending on the selected representation.

Evaluation focus

Record uplink and downlink payload sizes in bytes for each round, then compare the total communication cost alongside prediction accuracy.

Handling of Non-IID Data

Client drift and convergence

Baseline scenario

Intersections can have different traffic patterns, such as residential versus commercial peaks. Under non-IID data, standard FedAvg can experience client drift, unstable training gradients, and slow convergence.

Optimized scenario

FedProx adds a proximal term to the local objective function. This constrains local updates and is intended to make convergence steadier and more predictable.

Evaluation focus

Compare convergence behavior and prediction metrics under both IID and non-IID data distributions.

Communication Frequency

Local epochs and client sampling

Baseline scenario

Frequent synchronization can occur after every local epoch or after minimal local computation, causing repeated parameter exchanges.

Optimized scenario

Adaptive local epoch scaling allows clients to complete more local training before global synchronization. Selective client sampling uses a representative subset of virtual intersections in a round.

Evaluation focus

Track the number of communication rounds and the total bytes transmitted, while checking that prediction performance remains acceptable.

Network Overhead Cost

Uplink/downlink communication

Baseline scenario

Frequent, uncompressed updates produce high cumulative uplink and downlink traffic and increase the communication burden on cellular or edge backhaul.

Optimized scenario

Quantization, regularization, extended local training, and selective sampling are intended to reduce network expenditure and improve efficiency in bandwidth-constrained smart-city deployments.

Evaluation focus

Report total communication bytes together with MAE, RMSE, and convergence speed.

Optimization 1: Model Weight Quantization

Reducing serialized model payloads

Description

Convert 32-bit floating-point parameters (float32) to 16-bit floating-point values (float16) or 8-bit integers (int8) before serialization.

Impact

Reduces the byte size of uplink and downlink payloads. The comparison table estimates a 50%–75% reduction in transmission size.

Measurement

Measure actual serialized payloads in bytes rather than assuming the theoretical size reduction.

Optimization 2: FedProx Integration

Algorithmic regularization

Description

Add a proximal term to the local objective function to limit model drift across heterogeneous, non-IID intersections.

Impact

Aims to stabilize convergence and reduce the total number of communication rounds required.

Measurement

Compare convergence and prediction performance against the standard FedAvg baseline for IID and non-IID scenarios.

Optimization 3: Adaptive Local Epoch Scaling

Adjusting local training

Description

Allow edge clients to run multiple local training epochs before initiating global synchronization.

Impact

Reduces communication frequency and can help limit network congestion.

Measurement

Log local epochs, global rounds, total communication bytes, and the prediction metrics for each experiment.

Optimization 4: Selective Client Sampling

Choosing clients per round

Description

Involve a representative subset of virtual intersections per training round instead of requiring all nodes to transmit updates simultaneously.

Impact

Lowers concurrent server-side bandwidth pressure and can speed up iteration loops.

Measurement

Record which clients participate in each round and calculate the corresponding uplink/downlink traffic.

Experimental Setup

Multi-client federated traffic forecasting

Client architecture

Simulate 3 to 5 virtual traffic-intersection nodes connected to a central aggregation server; no physical hardware is required.

Data scenarios

Use an IID setup as a baseline and a non-IID setup that reflects localized traffic differences, such as morning rush-hour peaks versus night-time lulls.

Repeatability

Run experiments across 3 random seeds and compare accuracy, communication cost, and convergence speed.

Communication-Cost Evaluation

Byte tracking and comparison

Payload tracker

Integrate a serialization payload tracker to measure exact uplink and downlink network traffic in bytes per training round.

Joint assessment

Correlate network expenditure with prediction accuracy and convergence speed rather than evaluating communication savings alone.

Reported metrics

Log MAE, RMSE, and network overhead for baseline and optimized workflows.

Baseline and Optimized Workflow

Summary of the comparison

Baseline

Standard FedAvg with uncompressed float32 weights, frequent synchronization, and no proximal constraint on local updates.

Optimized workflow

Quantized weights, FedProx regularization, adaptive local epoch scaling, and selective client sampling.

Goal

Evaluate whether communication cost can be lowered while retaining useful traffic-flow prediction performance under both IID and non-IID conditions.

Interpretation Notes

Reading the comparison fairly

Payload size

The 50%–75% reduction is an estimate stated in the project comparison, not a guaranteed result for every model or serialization format.

Prediction quality

Any communication reduction should be assessed alongside prediction accuracy and convergence; the optimized workflow must be tested empirically.

Deployment context

The intended use case is a smart-city setting with distributed intersections and potentially bandwidth-constrained edge/backhaul links.

Solution Architecture & Design

1. Adamya Srivastava A3-PS003-TC517 October 14,2026

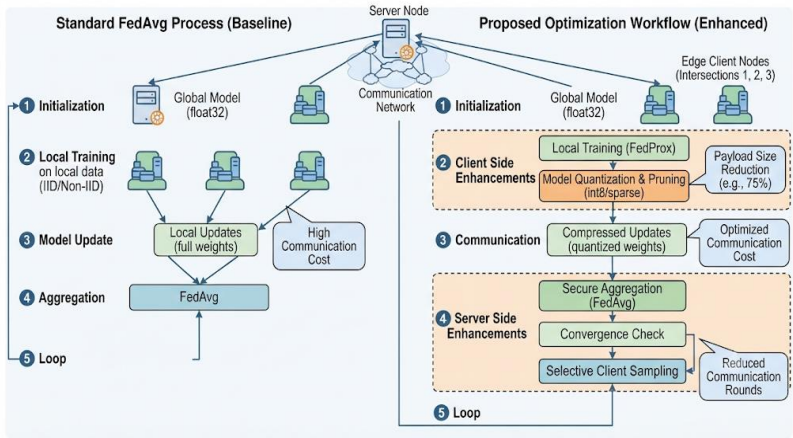

Federated Traffic-Flow Prediction: Standard vs. Optimized Workflow
Standard FedAvg Process (Baseline)
Server Node
Communication Network
Proposed Optimization Workflow (Enhanced)
Edge Client Nodes (Intersections 1, 2, 3)
1 Initialization
Global Model (float32)
2 Local Training on local data (IID/Non-IID)
3 Model Update
Local Updates (full weights)
High Communication Cost
4 Aggregation
FedAvg
5 Loop
1 Initialization
Global Model (float32)
2 Client Side Enhancements
Local Training (FedProx)
Model Quantization & Pruning (int8/sparse)
Payload Size Reduction (e.g., 75%)
3 Communication
Compressed Updates (quantized weights)
Optimized Communication Cost
4 Server Side Enhancements
Secure Aggregation (FedAvg)
Convergence Check
Selective Client Sampling
Reduced Communication Rounds
5 Loop

Figure: System Architecture UML Diagram

Data Flow Diagram

Overview of the architecture. Include the flow chart diagram created earlier. Visual: Diagrams or architecture visuals.

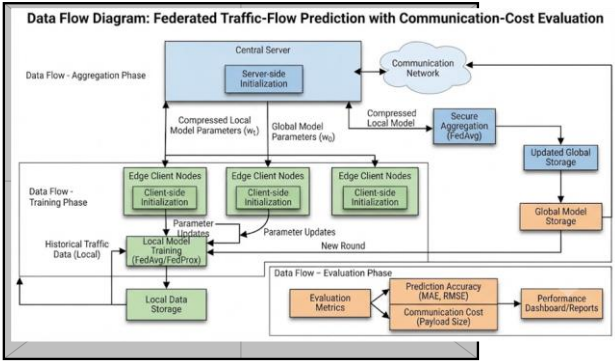


Figure: Data Flow Diagram (DFD)

Deployment Timeline

- **Week 1: Initialize the GitHub repository, set up the development environment, and acquire the traffic dataset.**
- **Week 2: Implement chronological data splitting to prevent data leakage and partition the data into 3 to 5 virtual intersections for both IID and non-IID distributions.**
- **Week 3: Develop the lightweight local forecasting model and verify individual client training pipelines.**

1. Adamya Srivastava A3-PS003-TC517 October 14,2026

Week 4: Code the standard Federated Averaging (FedAvg) server-client loop and integrate communication-cost byte tracking to complete baseline experiments.

Week 5: Implement advanced optimizations, including model weight quantization (converting float32 to int8/float16) and FedProx regularization to counter non-IID client drift.

Week 6: Run full experimental batches across 3 random seeds for both baseline and optimized workflows to log MAE, RMSE, and network overhead.

Week 7: Analyze the experimental results, generate comparative performance visualizations, and draft the final project report.

Week 8: Clean up the codebase, finalize repository documentation, transfer ownership to the hackathon account, and prepare for the mentor review.

Bibliography I

1] Abboud, C., & Khalil, S. (2026). FedTraffic: A hierarchical federated learning framework for traffic flow prediction in intelligent transportation systems. *Applied Sciences*, *7*(8), 362.

2] Li, Y. (2025). G-FedProx: Multi-factor traffic flow prediction model based on federated learning. *Proceedings of the 2025 3rd International Conference on Image, Algorithms, and Artificial Intelligence*

3] Orozco, F., de Gusmão, P. P. B., Wen, H., Wahlström, J., & Luo, M. (2024). Federated learning for traffic flow prediction with synthetic data augmentation. arXiv.

4] Villani, M. J., Natale, E., & Mallmann-Trenn, F. (2025). Trading-off accuracy and communication cost in federated learning.

Thank You for Your Attention!